

# Object Oriented Programming I

Dr. Burak Kaleci

September 27, 2019

# Content

Typical C++ Program Development Environment

A Simple C++ Program: Printing a Line of Text

Another C++ Program: Adding Integers

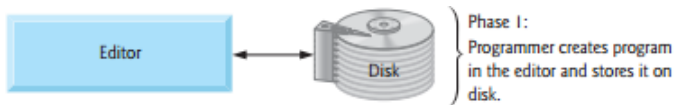
Memory Concepts

Arithmetic

Decision Making: Equality and Relational Operators

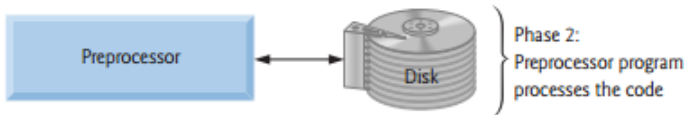
## Creating a Program

- ▶ C++ programs typically go through six phases to be executed.
- ▶ The first phase consists of editing a file. This is accomplished with an editor program.
- ▶ You type a C++ program with the editor, make corrections if necessary, then store the program on a secondary storage device such as a hard disk.
- ▶ C++ program file names should end with the .cpp, .cxx, .cc extensions.



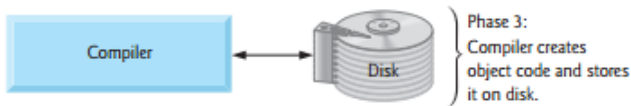
## Preprocessing a Program

- ▶ In Phase 2, the you give the command to compile the program.
- ▶ In a C++ system,a preprocessor program executes automatically before the compiler's translation phase begins
- ▶ The C++ preprocessor obeys commands called preprocessor directives, which indicate that certain manipulations are to be performed on the program before compilation.
- ▶ These manipulations usually include other text files to be compiled, and perform various text replacements.



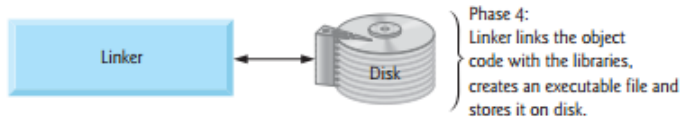
## Compiling a Program

- ▶ In Phase 3, you give the command to compile the program.
- ▶ The compiler translates the C++ program into machine language-code (also referred to as object code).

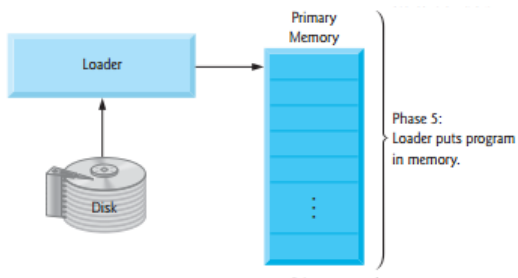


## Linking a Program

- ▶ C++ programs typically contain references to functions defined elsewhere, such as in the standard libraries or in the private libraries of groups of programmers working on a particular project.
- ▶ The object code produced by the C++ compiler typically contains "holes" due to these missing parts.
- ▶ A linker links the object code with the code for the missing functions to produce an executable program.



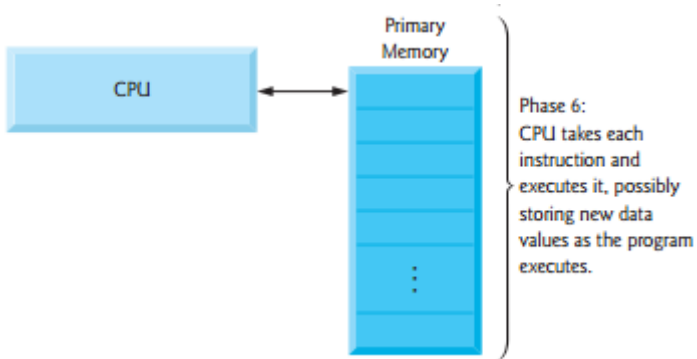
## Loading a Program



- ▶ Before a program can be executed, the program must first be placed in memory.
- ▶ This is done by the loader, which takes the executable image from disk and transfers it to memory.
- ▶ Additional components from shared libraries that support the program are also loaded.

## Execution

Finally, the computer, under the control of its CPU, executes the program one instruction at a time.



# Printing a Line of Text

```
firstProgram.cpp
1  /* *****
2  * Filename      : firstProgram.cpp
3  * Author       : Burak Kaleci
4  * Date        : 24.09.2018
5  * Description  : An example program to print a line of text
6  * *****/
7
8  // includes input/output stream header
9  // To output data to the screen
10 #include <iostream>
11
12 /* function main begins program execution
13    each program must include main function */
14 int main(void)
15 {
16     std::cout << "Welcome to C++!\n"; // display messages
17
18     return 0; // indicates successful termination
19 }
```

## Comments

- ▶ Line 8 and 9 begin with `//` indicating that these two lines are **comments**.
- ▶ You insert comments to document programs and improve program readability.
- ▶ Comments do not cause the computer to perform any action when the program is run.
- ▶ Comments are ignored by the C compiler and do not cause any machine-language object code to be generated.
- ▶ A comment beginning with `//` is called a single-line comment because it terminates at the end of the current line.
- ▶ You can also use C's style multi-line comments `/**/` in which everything from `/*` on the first line to `*/` at the end of the last line is a comment.

## #include Preprocessor Directive

- ▶ Line 10, `#include <iostream>`, is a preprocessor directive, which is a message to the C++ preprocessor.
- ▶ Lines that begin with `#` are processed by the preprocessor before the program is compiled.
- ▶ This line notifies the preprocessor to include in the program the contents of the input/output stream header `<iostream>`.
- ▶ This header must be included for any program that outputs data to the screen or inputs data from the keyboard using C++'s stream input/output.

## Blank Lines and White Space

- ▶ Line 7 is simply a blank line.
- ▶ You use blank lines, space characters and tab characters (i.e., "tabs") to make programs easier to read.
- ▶ Together, these characters are known as white space.
- ▶ White-space characters are normally ignored by the compiler.

# The **main** Function

- ▶ Line 14, `int main( void )`, is a part of **every C++ program**.
- ▶ The parentheses after **main** indicate that main is a program building block called a **function**.
- ▶ C++ programs typically consist of one or more functions and classes.
- ▶ Exactly one function in every program must be named **main**.
- ▶ C++ programs begin executing at function main, even if main is not the first function in the program.

## The **main** Function

- ▶ The keyword **int** to the **left of main** indicates that main "returns" an integer value.
- ▶ Functions also can **receive** information when they are called upon to execute.
- ▶ The **void** in parentheses here means that main **does not receive any information**.
- ▶ A left brace, { , begins the body of every function (line 15). A corresponding right brace, }, ends each function's body (line 19).
- ▶ This pair of braces and the portion of the program between the braces is called a block.
- ▶ The block is an important program unit in C.

## An Output Statement

- ▶ Line 16 `std::cout << "Welcome to C++!\n";` instructs the computer to perform an action, namely to print on the screen the string of characters marked by the quotation marks.
- ▶ The entire line, including `std::cout`, the `<<` operator, the string `"Welcome to C++!\n"` and the semicolon(`;`), is called a **statement**.
- ▶ Every C++ statement must end with a **semicolon** (also known as the statement terminator).
- ▶ Output and input in C++ are accomplished with streams of characters. Thus, when the preceding statement is executed, it sends the stream of characters `Welcome to C++!\n` to the standard output stream object (`std::cout`) which is normally "connected" to the screen.

## The **std** Namespace

- ▶ The `std::` before `cout` is required when we use names that we've brought into the program by the preprocessor directive `#include <iostream>`.
- ▶ The notation `std::cout` specifies that we are using a name, in this case `cout`, that belongs to "namespace" **std**.
- ▶ The names `cin` (the standard input stream) and `cerr` (the standard error stream) also belong to namespace `std`.
- ▶ You should simply remember to include `std::` before each mention of `cout`, `cin` and `cerr` in a program.
- ▶ This can be cumbersome, we introduce the **using** directive, which will enable you to omit `std::` before each use of a name in the `std` namespace.

# The Stream Insertion Operator

- ▶ The << operator is referred to as the stream insertion operator.
- ▶ When this program executes, the value to the operator's right is inserted in the output stream.
- ▶ Notice that the operator points in the direction of where the data goes.
- ▶ The right operand's characters normally print exactly as they appear between the double quotes.

## Escape Sequences

- ▶ However, the characters `\n` are not printed on the screen.
- ▶ The backslash (`\`) is called an escape character. It indicates that a "special" character is to be output.
- ▶ When encountering a backslash in a string, the compiler looks ahead at the next character and combines it with the backslash to form an escape sequence.
- ▶ The escape sequence `\n` means newline. It causes the cursor (i.e., the current screen-position indicator) to move to the beginning of the next line on the screen.

# Escape Sequences

| Escape sequence | Description                                                                                                        |
|-----------------|--------------------------------------------------------------------------------------------------------------------|
| <code>\n</code> | Newline. Position the screen cursor to the beginning of the next line.                                             |
| <code>\t</code> | Horizontal tab. Move the screen cursor to the next tab stop.                                                       |
| <code>\r</code> | Carriage return. Position the screen cursor to the beginning of the current line; do not advance to the next line. |
| <code>\a</code> | Alert. Sound the system bell.                                                                                      |
| <code>\\</code> | Backslash. Used to print a backslash character.                                                                    |
| <code>\'</code> | Single quote. Use to print a single quote character.                                                               |
| <code>\"</code> | Double quote. Used to print a double quote character.                                                              |

## Adding Two Integers

**Problem:** Write a C++ program which adds two integer numbers and prints the sum on the screen.

- Our next program uses the input stream object `std::cin` and the stream extraction operator, `>>`, to obtain two integers typed by a user at the keyboard, computes the sum of these values and outputs the result using `std::cout`.

```
1  /* *****  
2  * Filename   : add.cpp  
3  * Author    : Burak Kaleci  
4  * Date      : 24.09.2018  
5  * Description : An example program to sum two integer numbers  
6  * *****/  
7  
8  // includes input/output stream header  
9  // To output data to the screen  
10 #include <iostream>  
11  
12 /* function main begins program execution  
13    each program must include main function */  
14 int main(void)  
15 {  
16     int number1;    // first number to be entered by user  
17     int number2;    // first number to be entered by user  
18     int sum;        // variable in which sum will be stored  
19  
20     std::cout << "Enter first number:\n";    //prompt  
21     std::cin >> number1;    // read an integer  
22  
23     std::cout << "Enter second number:\n";    //prompt  
24     std::cin >> number2;    // read an integer  
25  
26     sum=number1+number2;    // assign total to sum  
27  
28     std::cout << "Sum is " << sum << std::endl;    // print sum  
29  
30     return 0;    // indicates successful termination  
31 }  
32
```

# Variable Declarations

- ▶

```
int number1;  
int number2;  
int sum;
```

are called as **declarations**.
- ▶ The identifiers **number1**, **number2** and **sum** are the names of **variables**.
- ▶ Variables are **locations in memory** where **values can be stored** for use by a program.
- ▶ These declarations specify that variables **number1**, **number2** and **sum** are data of **type int**, which means that **they will hold integer values**.

# Variable Declarations

- ▶ **All variables must be declared with a name and a data type before they can be used in a program.**
- ▶ Several variables of the same type may be declared in one declaration or in multiple declarations.
- ▶ We could have declared all three variables in one declaration by using a comma-separated list as follows:

```
int number1,number2,sum;
```

This makes the program less readable and prevents us from providing comments that describe each variable's purpose.

# Identifiers

- ▶ An identifier is a series of characters consisting of letters, digits, and \_ (underscore).
- ▶ Never start with numbers.
- ▶ Must be different than keywords (such as main, int, void).
- ▶ Case sensitive (a1 and A1 are different variable names).
- ▶ Variable names can be up to 31 characters.
- ▶ No Turkish specific letters!
- ▶ meaningful variable names helps make a program self-documenting (needs fewer comments).

## Obtaining the First Value from the User

- ▶ `std::cin >> number1;`  
uses the standard input stream object `cin` (of namespace `std`) and the stream extraction operator, `>>`, to obtain a value from the keyboard.
- ▶ Using the stream extraction operator with `std::cin` takes character input from the standard input stream, which is usually the keyboard.
- ▶ We like to pronounce the preceding statement as, "std::cin gives a value to number1" or simply "std::cin gives number1."

## Obtaining the First Value from the User

- ▶ When the computer executes the preceding statement, it waits for the user to enter a value for variable number1.
- ▶ The user responds by typing an integer, then pressing the Enter key to send the characters to the computer.
- ▶ The computer converts the character representation of the number to an integer and assigns this number to the variable number1.

## Displaying the Result

- ▶ `std::cout << "Sum is " << sum << std::endl;`  
displays the character string Sum is followed by the numerical value of variable sum followed by `std::endl` which is called stream manipulator.
- ▶ The name `endl` is an abbreviation for "endline" and belongs to namespace `std`.
- ▶ The preceding statement outputs multiple values of different types.
- ▶ The stream insertion operator "knows" how to output each type of data.

# Memory Concepts

- ▶ Variable names such as `number1`, `number2` and `sum` actually correspond to locations in the computer's memory.
- ▶ Every variable has **a name**, **a type** and **a value**.
- ▶ `std::cin >> number1;`  
when this statement is executed, the integer typed by the user is placed into a memory location to which the name `number1` has been assigned by the compiler.
- ▶ Suppose the user enters the number 45 as the value for `number1`. The computer will place 45 into location `number1`.

`number1`

45

## Memory Concepts

- ▶ Also, we can assign a value to number2 variable by using assignment operator (=) such as number2=72;
- ▶ Once the program has obtained values for number1 and number2, it adds these values and places the total into variable sum.

sum= number1+number2;

number2      72

- ▶ The previous statement that performs the addition **also replaces whatever value was stored in sum.**

|         |     |
|---------|-----|
| number1 | 45  |
| number2 | 72  |
| sum     | 117 |

# Arithmetic

Most programs perform arithmetic calculations. The C++ arithmetic operators are summarized in the following table

| C++ operation  | C++ arithmetic operator | Algebraic expression                   | C++ expression     |
|----------------|-------------------------|----------------------------------------|--------------------|
| Addition       | +                       | $f + 7$                                | <code>f + 7</code> |
| Subtraction    | -                       | $p - c$                                | <code>p - c</code> |
| Multiplication | *                       | $bm$ or $b \cdot m$                    | <code>b * m</code> |
| Division       | /                       | $x / y$ or $\frac{x}{y}$ or $x \div y$ | <code>x / y</code> |
| Modulus        | %                       | $r \bmod s$                            | <code>r % s</code> |

- ▶ In algebra, to multiply a times b, as in  $ab$ .
- ▶ In C++, however, if we were to do this,  $ab$  would be a variable name.
- ▶ Therefore, C++ require that multiplication be explicitly denoted by using the `*` operator, as in `a * b`.

## Integer Division and the Modulus Operator

- ▶ Integer division yields an integer result.
- ▶ For example, the expression  $7 / 4$  evaluates to 1 and the expression  $17 / 5$  evaluates to 3.
- ▶ Any fractional part in integer division is discarded (i.e., truncated) no rounding occurs.
- ▶ C++ provides the modulus operator, %, which yields the remainder after integer division.
- ▶ The % is an integer operator that can be used only with integer operands.
- ▶ The expression  $x \% y$  yields the remainder after  $x$  is divided by  $y$ .
- ▶ Thus,  $7 \% 4$  yields 3 and  $17 \% 5$  yields 2.

## Arithmetic Expressions in Straight-Line Form

- ▶ Arithmetic expressions in C++ must be written in straight-line form.
- ▶ Thus, expressions such as "a divided by b" must be written as  $a/b$  so that all operators and operands appear in a straight line.
- ▶ Parentheses are used in C++ expressions in the same manner as in algebraic expressions.
- ▶ For example, to multiply a times the quantity  $b + c$  we write  $a * (b + c)$ .

# Rules of Operator Precedence

C++ applies the operators in arithmetic expressions in a precise sequence determined by the following rules of operator precedence, which are generally the same as those in algebra:

## 1. Parentheses

- ▶ Operators in expressions contained within pairs of parentheses are evaluated first.
- ▶ **Thus, parentheses may be used to force the order of evaluation to occur in any sequence you desire.**
- ▶ Parentheses are said to be at the "**highest level of precedence.**"
- ▶ In cases of nested, or embedded, parentheses, such as  $((a + b) + c)$  the operators in the innermost pair of parentheses are applied first.

# Rules of Operator Precedence

## 2. Multiplication, division and modulus operations

- ▶ Multiplication, division and modulus are said to be on the same level of precedence.
- ▶ If an expression contains several multiplication, division and modulus operations, evaluation proceeds from left to right.

## 3. Addition and subtraction

- ▶ Addition and subtraction also have the same level of precedence, which is lower than the precedence of the multiplication, division and modulus operations.
- ▶ If an expression contains several addition and subtraction operations, evaluation proceeds from left to right.

## 4. The assignment operator (=)

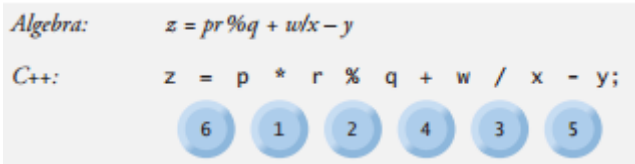
## Sample Algebraic and C++ Expressions

$$\text{Algebra:} \quad m = \frac{a + b + c + d + e}{5}$$
$$\text{C++:} \quad m = ( a + b + c + d + e ) / 5;$$

- ▶ The parentheses are required to group the additions because division has higher precedence than addition.
- ▶ The entire quantity ( a + b + c + d + e ) should be divided by 5.
- ▶ If the parentheses are erroneously omitted, we obtain a + b + c + d + e / 5, which evaluates incorrectly as

$$a + b + c + d + \frac{e}{5}$$

## Sample Algebraic and C++ Expressions



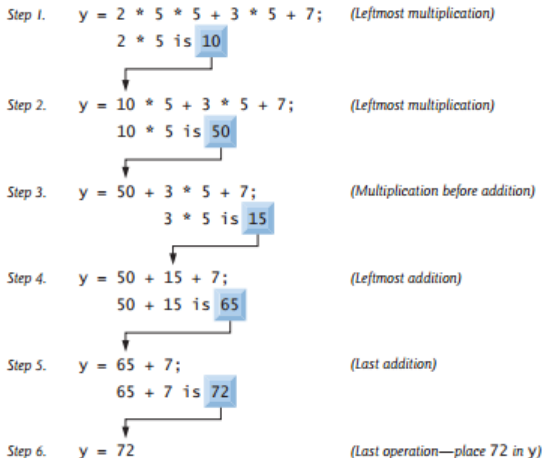
- ▶ The circled numbers indicate the order in which C++ evaluates the operators.
- ▶ The multiplication, modulus and division are evaluated first in left-to-right order because they have higher precedence than addition and subtraction.
- ▶ The addition and subtraction are evaluated next. They're also evaluated left to right.
- ▶ Finally, the result is assigned to the variable z.

## Sample Algebraic and C++ Expressions

The diagram illustrates the order of operations for the C++ expression `y = a * x * x + b * x + c;`. The expression is shown in a light gray box. Below each token in the expression, there is a blue circle containing a number indicating the order of evaluation. The tokens and their corresponding order numbers are: `y` (1), `=` (2), `a` (3), `*` (4), `x` (5), `*` (6), `x` (7), `+` (8), `b` (9), `*` (10), `x` (11), `+` (12), and `c;` (13). The circles are arranged in a row below the expression, with the numbers 6, 1, 2, 4, 3, and 5 visible under the first six tokens respectively.

- ▶ The circled numbers under the statement indicate the order in which C++ performs the operations. Suppose variables `a`, `b`, `c` and `x` in the preceding second-degree polynomial are initialized as follows: `a=2`, `b=3`, `c=7` and `x=5`.

## Sample Algebraic and C++ Expressions



## Equality and Relational Operators

In order to implement conditions, equality and relational operators are used. These operators are given in following table:

| Standard algebraic equality or relational operator | C++ equality or relational operator | Sample C++ condition | Meaning of C++ condition        |
|----------------------------------------------------|-------------------------------------|----------------------|---------------------------------|
| <i>Relational operators</i>                        |                                     |                      |                                 |
| >                                                  | >                                   | x > y                | x is greater than y             |
| <                                                  | <                                   | x < y                | x is less than y                |
| ≥                                                  | >=                                  | x >= y               | x is greater than or equal to y |
| ≤                                                  | <=                                  | x <= y               | x is less than or equal to y    |
| <i>Equality operators</i>                          |                                     |                      |                                 |
| =                                                  | ==                                  | x == y               | x is equal to y                 |
| ≠                                                  | !=                                  | x != y               | x is not equal to y             |

- ▶ The relational operators all have the same level of precedence and they associate left to right.
- ▶ The equality operators have a lower level of precedence than the relational operators and they also associate left to right.

## An example program for if statement

```
1  /* *****  
2  * Filename   : compare.cpp  
3  * Author    : Burak Kaleci  
4  * Date      : 24.09.2018  
5  * Description : An example program for if statement  
6  * *****  
7  
8  // includes input/output stream header  
9  // To output data to the screen  
10 #include <iostream>  
11  
12 using std::cout; // program uses cout  
13 using std::cin;  // program uses cin  
14 using std::endl; // program uses endl  
15  
16 /* function main begins program execution  
17    each program must include main function */  
18 int main(void)  
19 {  
20     int number1; // first number to be entered by user  
21     int number2; // second number to be entered by user  
22  
23     cout << "Enter two integers to compare:" << endl; //prompt  
24     cin >> number1 >> number2; // read two integers from user  
25  
26     if (number1 == number2)  
27     |     cout << number1 << "==" << number2 << endl;  
28  
29     if (number1 != number2)  
30     |     cout << number1 << "!=" << number2 << endl;  
31  
32     if (number1 < number2)  
33     |     cout << number1 << "<" << number2 << endl;  
34  
35     if (number1 > number2)  
36     |     cout << number1 << ">" << number2 << endl;  
37  
38     if (number1 <= number2)  
39     |     cout << number1 << "<=" << number2 << endl;  
40  
41     if (number1 >= number2)  
42     |     cout << number1 << ">=" << number2 << endl;  
43  
44     return 0; // indicates successful termination  
45 }  
46
```

## using Directives

- ▶ Lines 12-14 are **using** directives that eliminate the need to repeat the `std::` prefix as we did in earlier programs.
- ▶ We can now write `cout` instead of `std::cout`, `cin` instead of `std::cin` and `endl` instead of `std::endl`, respectively, in the remainder of the program.
- ▶ In place of lines 12-14, many programmers prefer to use the directive  
**using namespace std;**  
which enables a program to use all the names in any standard C++ header (such as `<iostream>`) that a program might include.